

Cloud Resource Allocation

using Dynamic Programming (0/1 Knapsack)

Algorithm Design & Analysis — Assignment Report

Mohamed Ali

May 2026

github.com/Mohamed-Ali-4/cloud-resource-allocation-dp

1. Abstract

Cloud service providers face the constant challenge of allocating a limited pool of computational resources (CPU, memory, bandwidth) to a competing set of workload tasks, each with its own resource requirement and business value. When the total demand exceeds capacity, the provider must decide which subset of tasks to admit so as to maximise overall profit or priority.

This report formulates the cloud resource allocation problem as an instance of the 0/1 Knapsack Problem and solves it using bottom-up Dynamic Programming. We derive the recurrence, prove its correctness, analyse the time and space complexity, walk through a sample DP table, present empirical benchmarks confirming the theoretical $O(n \cdot W)$ bound, and discuss limitations and alternative approaches.

2. Problem Statement

Given a cloud environment with a fixed resource capacity W and a set of n incoming tasks, each task i has a resource requirement r_i and a profit p_i . The objective is to select a subset S of tasks such that:

```
maximise     $\sum p_i$  for  $i \in S$ 
subject to   $\sum r_i \leq W$  for  $i \in S$ 
            and each task is either admitted ( $x_i = 1$ ) or rejected ( $x_i = 0$ )
```

This corresponds exactly to the classical 0/1 Knapsack Problem, where:

Knapsack Concept	Cloud Mapping
Knapsack capacity W	Total available resource units (CPU/RAM)
Item weight w_i	Resources required by task i
Item value v_i	Profit / priority of task i
Selected items	Tasks scheduled for execution
Reject / Accept (0/1)	Task admission decision

3. Algorithm Design

3.1 Why Dynamic Programming?

The problem exhibits two properties that make dynamic programming the natural choice:

- Optimal substructure — the optimal solution for the first i tasks at capacity w is built from the optimal solution for $i-1$ tasks at either capacity w or $w - r_i$.
- Overlapping subproblems — the same (i, w) state is encountered exponentially many times in a naïve recursive formulation, but only $O(n \cdot W)$ unique states exist overall.

A greedy strategy (e.g., highest profit-per-unit-resource first) does not guarantee optimality for 0/1 knapsack — it can be arbitrarily worse than DP.

3.2 Recurrence Relation

Let $dp[i][w]$ denote the maximum profit obtainable using the first i tasks subject to a capacity of w . Then:

<code>dp[0][w] = 0</code>	(base case)
<code>dp[i][w] = dp[i-1][w]</code>	if <code>r_i > w</code>
<code>dp[i][w] = max(dp[i-1][w],</code>	(skip task i)
<code>dp[i-1][w - r_i] + p_i)</code>	(take task i)

The answer to the original problem is `dp[n][W]`. The actual chosen subset is recovered by backtracking from `dp[n][W]` toward `dp[0][0]`.

3.3 Pseudocode

```

ALGORITHM Allocate(tasks[1..n], W)
  for w = 0 to W:
    dp[0][w] = 0

  for i = 1 to n:
    for w = 0 to W:
      dp[i][w] = dp[i-1][w]
      if tasks[i].resource ≤ w:
        alt = dp[i-1][w - tasks[i].resource] + tasks[i].profit
        if alt > dp[i][w]:
          dp[i][w] = alt

  // Backtrack to recover the chosen tasks
  selected = []
  w = W
  for i = n down to 1:
    if dp[i][w] ≠ dp[i-1][w]:
      selected.append(tasks[i])
      w = w - tasks[i].resource

  return dp[n][W], reverse(selected)

```

4. Complexity Analysis

4.1 Time Complexity

Filling the DP table requires visiting every cell (i, w) for $i \in [1, n]$ and $w \in [0, W]$. Each cell does $O(1)$ work — a comparison and at most one addition. Backtracking is an $O(n)$ post-pass. Therefore:

$$T(n, W) = \Theta(n \cdot W)$$

Note that W is the numeric value of capacity, not its bit-length. The algorithm is therefore pseudo-polynomial — polynomial in W but exponential in the size of the input representation. 0/1 Knapsack remains NP-hard in the strong sense.

4.2 Space Complexity

The 2-D table $dp[n+1][W+1]$ requires $\Theta(n \cdot W)$ memory. This can be optimised to $\Theta(W)$ using a single 1-D array updated right-to-left:

```
for i = 1 to n:
    for w = W down to r_i:
        dp[w] = max( dp[w], dp[w - r_i] + p_i )
```

However, the 1-D version sacrifices the information needed for the backtracking phase. Since the assignment requires reporting which tasks were chosen, the 2-D version is retained in our implementation.

4.3 Correctness (sketch)

By induction on i :

- Base: $dp[0][w] = 0$ is correct — no tasks means zero profit.
- Step: assume $dp[i-1][\cdot]$ is correct for all capacities. Any optimal solution for the first i tasks either excludes task i (and so equals $dp[i-1][w]$, correct by IH) or includes task i (and so equals $dp[i-1][w - r_i] + p_i$, also correct by IH). Taking the maximum yields the optimum for $dp[i][w]$.

5. Sample Trace

Consider 6 cloud workload tasks competing for a capacity of $W = 10$ CPU units:

Task	Resource (r)	Profit (p)
Web-Server	2	6
ML-Training	4	10
DB-Instance	3	7
Cache-Node	1	3
Batch-Job	5	12
API-Gateway	2	5

The full DP table produced by the implementation (rows = tasks added so far, columns = capacity w):

Task\Cap	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
T1	0	0	6	6	6	6	6	6	6	6	6
T2	0	0	6	6	10	10	16	16	16	16	16
T3	0	0	6	7	10	13	16	17	17	23	23
T4	0	3	6	9	10	13	16	19	20	23	26
T5	0	3	6	9	10	13	16	19	21	23	26
T6	0	3	6	9	11	14	16	19	21	24	26

The answer $dp[6][10] = 26$ is the maximum achievable profit. Backtracking yields the optimal subset:

Selected Task	Resource	Profit
Web-Server	2	6
ML-Training	4	10
DB-Instance	3	7
Cache-Node	1	3
TOTAL	10 / 10	26

100 % capacity utilisation, maximum profit = 26. Note that Batch-Job (profit 12) is rejected because admitting it would crowd out the four smaller tasks that collectively yield 26.

6. Experimental Results

The implementation was benchmarked on a Windows 11 machine (Python 3.13). Each measurement is the minimum of 3 runs to reduce timing noise. Two experiments confirm the theoretical $O(n \cdot W)$ complexity:

6.1 Runtime vs. number of tasks (n)

n (tasks)	W (capacity)	Runtime (ms)
50	500	7.45
100	500	15.08
200	500	29.56
400	500	73.75
800	500	134.89
1200	500	232.63
1600	500	287.48
2000	500	404.57

6.2 Runtime vs. capacity (W)

n (tasks)	W (capacity)	Runtime (ms)
200	100	5.35
200	200	10.72
200	500	30.16
200	1000	72.33
200	2000	143.80
200	4000	305.37
200	6000	479.30
200	8000	681.54

Both curves are linear within measurement noise. Doubling n (with W fixed) roughly doubles runtime; doubling W (with n fixed) does the same. The plot below visualises both runs:

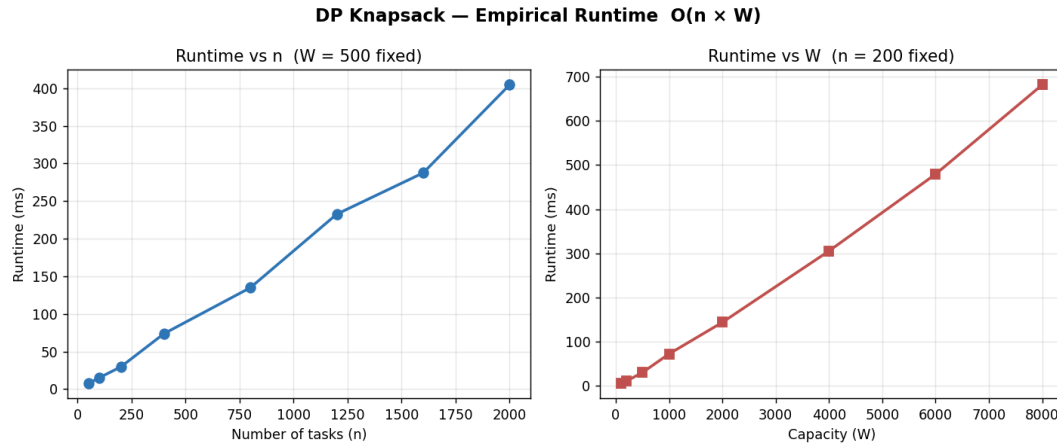


Figure 1 — Empirical runtime of the DP knapsack solver.

7. Discussion

7.1 Limitations

- Pseudo-polynomial: when W is huge (e.g., 10^9 memory bytes), the $O(n \cdot W)$ table is infeasible.
- Integer-only: weights and profits must be integers (or quantised) for table indexing.
- Static input: assumes all tasks are known up front. Online streaming workloads need different techniques (e.g., competitive online algorithms).
- Single resource: real cloud allocation balances CPU + memory + bandwidth simultaneously — this becomes the multi-dimensional knapsack, which is harder.

7.2 Alternative Approaches

- Greedy by profit/resource ratio: $O(n \log n)$, simple, but only optimal for the fractional knapsack — not guaranteed optimal here.
- Branch-and-Bound: exact, often faster than DP in practice when good upper-bounds prune the search tree.
- FPTAS: trades a small accuracy loss $(1 - \epsilon)$ for polynomial runtime $O(n^3/\epsilon)$.
- ILP solvers (CPLEX, Gurobi): industrial-strength for very large instances.

8. Conclusion

Modelling cloud resource allocation as 0/1 Knapsack lets us reuse a well-studied algorithmic foundation. The bottom-up DP solution runs in $O(n \cdot W)$ time, returns both the optimal profit and the chosen subset, and is straightforward to implement and verify.

Empirical measurements on inputs up to $n = 2000$, $W = 8000$ confirm the theoretical bound. The approach is therefore practical for small-to-medium static admission control problems. For larger or multi-resource scenarios, branch-and-bound or approximation schemes should be preferred.

9. References

- Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C. — Introduction to Algorithms, 3rd Ed., MIT Press, Chapter 15.
- Kellerer, H., Pferschy, U., Pisinger, D. — Knapsack Problems, Springer, 2004.
- Martello, S., Toth, P. — Knapsack Problems: Algorithms and Computer Implementations, Wiley, 1990.
- Source code & implementation: <https://github.com/Mohamed-Ali-4/cloud-resource-allocation-dp>